

CLASS-APP Learning Guide: Full Repository Explanation

Version: Based on CLASS-APP v1.6.4 (changelog v1.9.16, May 11 2026)

Audience: First-year IT students and beginner developers

Purpose: Understand how CLASS-APP is built, what every file does, and how all parts connect.

Table of Contents

1. [App Overview](#)
2. [File and Folder Explanation](#)
3. [HTML Explanation](#)
4. [CSS Explanation](#)
5. [JavaScript Explanation](#)
6. [Variables and Purposes](#)
7. [Functions and Purposes](#)
8. [Authentication Explanation](#)
9. [Hash, Salt, Password, and Security](#)
10. [Database and Supabase Explanation](#)
11. [Storage and File Uploads](#)
12. [AI Integrations](#)
13. [PWA, Cache, and Service Worker](#)
14. [Page-by-Page Feature Explanation](#)
15. [Button and Event Listener Map](#)
16. [Common Bug Patterns in the App](#)
17. [Glossary of Jargon](#)
18. [Learning Roadmap](#)
19. [Repository Health Check](#)
20. [Missing / Unable to Analyze](#)

1. App Overview

What is CLASS-APP?

CLASS-APP is a **school portfolio web application** built for college IT students. It runs in a browser (on phones, tablets, or computers) and can also be installed on a phone's home screen like a real app (this is called a **PWA** — Progressive Web App).

Think of it as a private school dashboard where students can:

- View and organize class files
- Read and share study reviewers

- Summarize uploaded documents using AI
- Chat with classmates in real time
- Play browser games between study sessions
- Listen to music
- Set alarms and keep notes
- Track class schedules and announcements

What type of app is it?

Type	Yes/No	Explanation
Web App	✓	Runs in a browser at a URL
PWA	✓	Can be installed on phones like an app
Mobile-First	✓	Designed primarily for phones
iOS + Android	✓	Works on both
Real-Time App	✓	Chat and lobby update instantly
AI-Powered	✓	File summaries and quizzes generated by AI
Multi-User	✓	Multiple students can log in

Main Features

1. **My Classes** — Year 1 to Year 4, First and Second Semester folders
2. **File Manager** — Upload, organize, view, and download files in folders
3. **File Summarizer** — Upload a PDF/DOCX/PPTX and get an AI summary or quiz
4. **Shared Reviewer** — Students share and vote on study notes
5. **AI Assistants** — Chat with Gemini or Groq AI models
6. **Coding Lessons** — Interactive coding lessons with a built-in code lab
7. **Chat** — Real-time group and private messaging via Socket.IO
8. **Calendar** — Personal calendar with sticky notes per date
9. **Music Hub** — Search and play YouTube music
10. **Games** — Pokémon battle, Battle Royale, Candy Match, Pac-Man, Tetris, Lobby platformer
11. **Personal Tools** — Alarm clock, Notepad, Scientific Calculator, Personalization
12. **User Directory** — See who is online
13. **Announcements** — Admin-posted announcements
14. **Social Media** — Embedded Facebook page view
15. **Software Update** — Changelog of every version update
16. **Admin Panel** — Admin-only tools (diagnostics, user management)

How the App is Connected (System Flow)

```

[Student's Phone/Browser]
|
| opens URL (e.g., https://class-app.onrender.com)
v
[index.html] ← loads CSS (style.css) + JS (script.js, feature files)
|
| on page load → fetches /api/config
v
[server.js – Express Backend on Render.com]
|
|— /api/login —————→ [bcrypt + JWT] → token stored in localStorage
|— /api/summarize-file —→ [Gemini AI / Groq AI] → returns summary
|— /api/yt-search —————→ [YouTube API] → music results
|— /uploads/* —————→ [Cloudflare R2] → files served
|— Socket.IO —————→ [Real-time chat and lobby events]
|
v
[Supabase – Cloud PostgreSQL Database]
|
|— folders, files, profiles tables ← queried directly from browser
|— chat messages, announcements ← queried via frontend JS
|— RLS policies — enforce who can see what at database level
|
[Cloudflare R2 – File Storage]
|— PDFs, images, videos, documents uploaded here
|— served via /uploads/* route in server.js

[AI Services]
|— Gemini 1.5 Flash (primary)
|— Groq (fallback if Gemini quota exceeded)
|— Local sentence extractor (fallback if both AI services fail)

```

2. File and Folder Explanation

Repository Structure (Full Inventory)

```

CLASS-APP/
├─ index.html           ─ Main app page (all UI is here)
├─ script.js           ─ Main frontend logic (7,571 lines)
├─ server.js           ─ Express backend (2,913 lines)
├─ sw.js               ─ Service Worker (PWA cache management)
├─ style.css           ─ Global styles (5,671 lines)
├─ manifest.json       ─ PWA install configuration
├─ package.json        ─ Project info + dependencies
├─ package-lock.json   ─ Exact dependency versions
├─ ai-service.js       ─ AI fallback logic (frontend)
├─ migrate-to-r2.js     ─ One-time migration script
├─ file-summarizer.css ─ Legacy CSS (root level, not main one)
├─ data.json           ─ Runtime data store
├─ dashboard-mockup.html ─ UI mockup (not used in production)
├─ .env.example        ─ Template for environment variables
├─ .gitignore          ─ Files Git should ignore
├─ .eslintrc.json      ─ JavaScript linting rules
├─ .eslintignore       ─ Files ESLint should ignore
├─ .dockerignore       ─ Files Docker should ignore
├─ Dockerfile          ─ Docker container build instructions
├─ render.yaml         ─ Render.com deployment configuration
├─ CLAUDE.md           ─ AI assistant instructions for this repo
├─ README.md           ─ Project introduction
├─ AGENTS.md           ─ Multi-agent coding rules
├─ app-hardening.md    ─ Security hardening documentation
├─ MUST-FIX            ─ Critical bug documentation
├─
├─ assets/
│  ├─ css/codelab.css   ─ Code Lab styling
│  ├─ js/codelab.js     ─ Code Lab interactive UI
│  └─ images/
│     ├─ code-web-card.svg ─ Web coding lesson card image
│     └─ code-java-card.svg ─ Java coding lesson card image
├─
├─ icons/
│  ├─ icon-192.png      ─ PWA icon (small, for app drawer)
│  └─ icon-512.png      ─ PWA icon (large, for splash screen)
├─
├─ scripts/
│  ├─ cache-bust.js     ─ Auto-bumps cache version before deploy
│  └─ version-check.js  ─ Validates file versions match between sw.js and index.html
├─
├─ coding-educational/
│  ├─ coding-educational.js ─ Interactive coding lesson engine
│  ├─ coding-educational-data.js ─ All lesson content/data (133KB)
│  ├─ coding-educational.css ─ Lesson UI styling
│  └─ assets/fallback-card.jpg ─ Placeholder image for lessons
├─
├─ features/           ─ 25 feature modules (each has its own JS/CSS)
│  ├─ academics/       ─ My Classes page (stub module)
│  ├─ ai/              ─ AI Assistants page (stub + config)
│  ├─ calendar/        ─ Calendar page (stub module)
│  ├─ candy/           ─ Candy Match game
│  ├─ chat/            ─ Chat page (stub module)
│  ├─ file-summarizer/ ─ File Summarizer tool
│  ├─ folders/         ─ Folder system (stub module)
│  ├─ gallery/         ─ Gallery page (stub module)
│  ├─ games/           ─ Games hub (stub module)
│  └─ hotfixes/        ─ Interaction safety guards

```

```

|   ├── lobby/                ← Lobby platformer game (stub module)
|   ├── logging-in/          ← Auth bootstrap + shell navigation
|   ├── music/               ← Music Hub (stub module)
|   ├── notepad/             ← Legacy notepad (replaced by personal-tools/notepad.js)
|   ├── pacman/              ← Pac-Man game
|   ├── personal-tools/      ← Tools: alarm, notepad, calculator, personalization
|   ├── personalization/     ← Background presets
|   ├── pokemon/             ← Pokémon battle game (161KB)
|   ├── reviewers/          ← Shared Reviewer page
|   ├── royale/              ← Battle Royale game (151KB)
|   ├── security/            ← Session manager + form validation
|   ├── social/              ← Social media embed
|   ├── tetris/              ← Tetris game
|   ├── updates/             ← Software Update page + changelog
|   └── users/               ← User Directory (stub module)
|
|   └── supabase/
|       ├── functions/
|           ├── check-alarms/
|               └── index.ts    ← Edge Function for alarm push notifications
|           └── migrations/    ← 28 SQL files that set up the database
|               ├── 001_functions_and_columns.sql
|               ├── 002_tables.sql
|               ├── 003_rls_policies.sql
|               └── ... (up to 028_tetris_leaderboard.sql)
|
|   └── tests/
|       └── server.test.js     ← Jest tests for the backend API

```

What Each File Type Means

HTML files (.html)

HTML is the skeleton of a webpage. It tells the browser "put a button here, put text here, make a box there."

File	Purpose
<code>index.html</code>	The entire app UI — all pages, buttons, modals, containers
<code>dashboard-mockup.html</code>	An early design mockup, not used in production

Example from this repo:

```
<button id="hamburger-btn" onClick="toggleMenu()">☰</button>
```

This creates the hamburger (three-line) menu button in the top corner.

CSS files (.css)

CSS is the paint and layout of a webpage. It controls colors, sizes, positions, animations.

File	Purpose
<code>style.css</code>	Global styles for everything
<code>file-summarizer.css</code> (root)	Legacy leftover — the real one is in <code>features/file-summarizer/</code>
<code>assets/css/codecademy.css</code>	Styling for the coding lessons
<code>coding-educational/coding-educational.css</code>	Lesson card layout
<code>features/*/feature-name.css</code>	Each feature's own private styles

JavaScript files (.js)

JavaScript makes the app interactive. Buttons click, pages open, data loads — all JavaScript.

File	Purpose
<code>script.js</code>	Main frontend logic — 7,571 lines, handles pages, auth, folders, chat
<code>server.js</code>	The backend (Node.js/Express) — runs on the server, handles API calls
<code>sw.js</code>	Service Worker — manages offline caching
<code>ai-service.js</code>	AI provider fallback chain (frontend helper)
<code>migrate-to-r2.js</code>	One-time migration script — NOT part of the running app
<code>scripts/cache-bust.js</code>	Automatically updates cache version before deploy
<code>scripts/version-check.js</code>	Validates versions are consistent

JSON files (.json)

JSON is a format for storing data. Think of it as organized information in text form.

File	Purpose
<code>package.json</code>	Lists the project name, version, scripts, and all dependencies
<code>package-lock.json</code>	Exact copy of every dependency version (do not edit manually)
<code>manifest.json</code>	Tells the browser how to install this app on a phone (PWA)
<code>data.json</code>	Small runtime data store (user counts, in-memory data)
<code>.eslintrc.json</code>	Rules for ESLint (catches code mistakes)

Markdown files (.md)

Markdown is a simple way to write formatted documents (like this guide).

File	Purpose
<code>README.md</code>	Short project introduction
<code>CLAUDE.md</code>	Detailed rules for AI assistants working on this repo
<code>AGENTS.md</code>	Multi-agent coding guidelines
<code>app-hardening.md</code>	Security hardening steps taken
<code>MUST-FIX</code>	A file (no extension) documenting a critical bug

SQL files (.sql)

SQL is the language for databases. These files create and configure the database tables.

Located in: `supabase/migrations/` Each file is a "migration" — a step that changes the database structure. They run in order: 001 → 002 → ... → 028.

TypeScript file (.ts)

TypeScript is JavaScript with added type checking. Used in:

- `supabase/functions/check-alarms/index.ts` — Supabase Edge Function for alarm push notifications

Config/Infrastructure files

File	Purpose
<code>.env.example</code>	Template showing what environment variables are needed
<code>.gitignore</code>	List of files Git should NOT track (e.g., <code>.env</code> with real secrets)
<code>Dockerfile</code>	Instructions for building a Docker container
<code>render.yaml</code>	Deployment settings for Render.com hosting

SVG files (.svg)

SVG (Scalable Vector Graphics) are image files that don't blur when resized. Used for lesson card icons.

PNG files (.png)

PNG images are used for:

- `icons/icon-192.png` — Small PWA icon (192×192 pixels)
- `icons/icon-512.png` — Large PWA icon (512×512 pixels)

3. HTML Explanation

What HTML Does in CLASS-APP

`index.html` is the **only HTML file the user sees**. It is 1,734 lines long and contains all the app's pages, buttons, sidebars, modals, and containers at once. JavaScript shows or hides each section depending on what page the user is on.

Main Sections of index.html

Splash / Loading Screen

```
<div id="splash-screen">
  
  <div class="loading-spinner"></div>
</div>
```

This is the logo+spinner that appears while the app loads. JavaScript hides it when loading completes.

Background Layer

```
<canvas id="stars-canvas"></canvas>
<div id="custom-bg-overlay"></div>
```

- `stars-canvas` — an animated starfield drawn with JavaScript
- `custom-bg-overlay` — where the user's chosen background (gradient or custom image) is applied

Navigation Sidebar

```
<nav id="sidebar" class="sidebar">
  <button id="hamburger-btn" onclick="toggleMenu()">☰</button>
  <ul id="nav-list">
    <li data-page="home" class="nav-item">Home</li>
    <li data-page="lobby" class="nav-item">Lobby</li>
    ...
  </ul>
</nav>
```

The sidebar contains all navigation links. Clicking a `nav-item` calls `goToPage()` in JavaScript.

Page Containers

```
<div id="home-page" class="page active">...</div>
<div id="lobby-page" class="page">...</div>
<div id="file-summarizer-page" class="page">...</div>
```

Every feature has its own `<div>` container with `class="page"`. Only the active page is visible. Pages are switched by JavaScript toggling the `active` class.

Modals (Popup Dialogs)

```
<div id="alert-modal" class="modal-overlay">
  <div class="modal-box">
    <p id="alert-message"></p>
    <button id="alert-ok-btn">OK</button>
  </div>
</div>
```

Instead of browser popup alerts (which look ugly), the app has custom modals for alerts, confirmations, and text input.

Audio Element

```
<audio id="notification-sound" src="..."></audio>
```

Used to play notification sounds when messages arrive.

Script Loading (at bottom of index.html)

```
<script src="https://cdn.socket.io/..."></script>
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js"></script>
<script src="features/logging-in/loading-components.js?v=13"></script>
<script src="features/logging-in/shell-controls.js?v=17"></script>
<script src="script.js?v=149"></script>
<script src="features/updates/changelog.js?v=52"></script>
...
```

Scripts are loaded at the bottom so the HTML renders first. The `?v=N` numbers force browsers to re-download files when they change.

Important HTML IDs and Classes

ID / Class	Element	Purpose
<code>#splash-screen</code>	div	Loading screen

ID / Class	Element	Purpose
<code>#sidebar</code>	nav	Navigation menu
<code>#hamburger-btn</code>	button	Opens/closes sidebar
<code>.page</code>	div	Page container (hidden by default)
<code>.page.active</code>	div	Currently visible page
<code>#home-page</code>	div	Homepage container
<code>#file-summarizer-page</code>	div	File Summarizer page
<code>#alert-modal</code>	div	Custom alert popup
<code>#confirm-modal</code>	div	Custom confirm popup
<code>#prompt-modal</code>	div	Custom input popup
<code>.nav-item</code>	li	Sidebar navigation links
<code>.modal-overlay</code>	div	Dark backdrop behind modals
<code>.modal-box</code>	div	White/glass card inside modal

4. CSS Explanation

What CSS Does in CLASS-APP

`style.css` (5,671 lines) styles every visual element. It controls:

- Colors and themes (glassmorphism, dark background)
- Layout (how elements are arranged)
- Animations (fade in, slide up, pulse effects)
- Responsiveness (how the app looks on phones vs. computers)

Main Style Sections

CSS Variables (Custom Properties)

```
:root {
  --mountain-color: #4a90d9;
  --ocean-color: #00b4d8;
  --bg-dark: #020617;
  --glass-bg: rgba(255, 255, 255, 0.05);
}
```

CSS variables let one color value be reused everywhere. Changing `--ocean-color` in one place updates every element using it.

Glassmorphism

The "glass card" effect seen throughout the app:

```
.card {
  background: rgba(255, 255, 255, 0.05);
  backdrop-filter: blur(12px);
  border: 1px solid rgba(255,255,255,0.1);
  border-radius: 16px;
}
```

This creates semi-transparent, frosted-glass looking boxes.

Flexbox Layout

```
.flex-row {
  display: flex;
  flex-direction: row;
  align-items: center;
  gap: 12px;
}
```

Flex is used to arrange buttons and cards side by side or in rows.

Grid Layout

Used for card grids (like the tools page or subject cards):

```
.cards-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(160px, 1fr));
  gap: 16px;
}
```

This creates a responsive grid where cards automatically wrap to the next line on smaller screens.

Media Queries (Responsive Design)

```
@media (max-width: 768px) {
  .sidebar {
    width: 100%;
  }
}
```

Media queries change the layout based on screen size. Below 768 pixels wide (typical phone width), the sidebar becomes full-width.

Z-index (Layer Control)

```
.sidebar { z-index: 100; }
.modal-overlay { z-index: 200; }
.splash-screen { z-index: 9999; }
```

Z-index controls which elements appear on top of others. Higher number = appears in front.

Important CSS Concepts Used in This App

Term	What it Means	Example Use in App
<code>class</code>	A reusable style label	<code>.nav-item</code> styles all sidebar links
<code>id</code>	A unique style label (one element only)	<code>#sidebar</code> styles the one sidebar

Term	What it Means	Example Use in App
<code>selector</code>	The target of a CSS rule	<code>.page.active</code> targets active pages only
<code>flex</code>	Layout method for rows/columns	Navigation bar, button groups
<code>grid</code>	Layout for 2D card grids	Tool cards, game cards, subject cards
<code>media query</code>	Rules that change at different screen sizes	Mobile layout adjustments
<code>z-index</code>	Controls which element appears on top	Modals above pages, splash above all
<code>position: fixed</code>	Sticks element to screen even while scrolling	Sidebar, loading screen, notification banner
<code>position: absolute</code>	Positions element relative to its parent	Tooltip, badge counter
<code>animation</code>	CSS-controlled movement over time	Pulse effects, fade in, slide up
<code>transition</code>	Smooth change when a style changes	Button color change on hover
<code>hover</code>	Style applied when mouse is over element	Button brightens on hover
<code>active</code>	Style applied when element is clicked	Button darkens on press
<code>glassmorphism</code>	Frosted-glass card effect	Feature cards, modals
<code>overlay</code>	Semi-transparent dark layer covering screen	Behind modals, blocking the background
<code>backdrop-filter: blur</code>	Blurs what's behind an element	Glass card effect
<code>pointer-events: none</code>	Makes element unclickable (doesn't intercept mouse/touch)	Hidden pages (so background buttons can't be clicked)

5. JavaScript Explanation

What JavaScript Does in CLASS-APP

JavaScript is what makes CLASS-APP a living, interactive application. Without JavaScript, the app would just be a static page that does nothing. JavaScript handles:

- Showing and hiding pages
- Buttons clicking and doing things
- Sending and receiving data from the server and database
- Chat messages appearing in real time
- AI summaries being generated
- Games running

Main JS Files

File	Lines	Purpose
<code>script.js</code>	7,571	Everything: pages, auth, folders, chat, UI helpers
<code>server.js</code>	2,913	Backend: API routes, auth, file processing, AI calls
<code>features/logging-in/loading-components.js</code>	~200	Supabase init, auth bootstrap, splash screen

File	Lines	Purpose
<code>features/logging-in/shell-controls.js</code>	~350	Sidebar, page switching, menu
<code>features/updates/changelog.js</code>	~3,000+	All version history (very large)
<code>features/file-summarizer/file-summarizer.js</code>	~1,000	File upload, AI summary, quiz
<code>features/reviewers/reviewers.js</code>	~700	Shared reviewer board
<code>features/personal-tools/notepad.js</code>	693	Cloud notepad with offline sync
<code>features/pokemon/pokemon.js</code>	~4,000	Pokémon battle game
<code>features/royale/royale.js</code>	~4,000	Battle Royale game
<code>features/candy/candy.js</code>	~2,000	Candy Match game

How Buttons Work

Buttons in CLASS-APP work in two ways:

Method 1: Inline onclick (older style)

```
<button onclick="toggleMenu()">☰</button>
```

The button calls the function directly in HTML. The function must be on the `window` object.

Method 2: Event Listener (modern style)

```
document.getElementById('save-btn').addEventListener('click', function() {
  saveNote();
});
```

The button is connected to a function in JavaScript.

⚠ Important Bug Pattern: If an inline onclick calls a function that is NOT attached to `window`, clicking the button does nothing (no error shown). This is a common bug in this app.

How Pages Open and Close

Pages are `<div>` elements with class `"page"`. Only one is visible at a time.

```
window.goToPage = function(pageName) {
  // 1. Hide all pages
  document.querySelectorAll('.page').forEach(p => {
    p.classList.remove('active');
  });

  // 2. Show the selected page
  document.getElementById(pageName + '-page').classList.add('active');

  // 3. Initialize the page's module if needed
  if (pageName === 'file-summarizer') {
    window.fileSummarizerModule?.init();
  }
};
```

How Modals Work

Instead of `alert()`, `confirm()`, and `prompt()` (browser default popups), the app uses custom modals:

```

window.customAlert = function(message) {
  document.getElementById('alert-message').textContent = message;
  document.getElementById('alert-modal').classList.add('active');
};

```

How Authentication Works in JS

1. User enters username + password on the login screen
2. JS sends `POST /api/login` to the backend
3. Backend returns a **JWT token** (a string of encrypted data)
4. JS saves the token: `localStorage.setItem('classAppToken', token)`
5. Every future request includes: `Authorization: Bearer <token>`

How localStorage / sessionStorage is Used

Key	Location	Purpose
<code>classAppToken</code>	localStorage	JWT auth token (persists across browser close)
<code>classAppUser</code>	localStorage	Current user's username
<code>notepad-notes</code>	localStorage	Offline notepad backup
<code>fs-quiz-settings</code>	localStorage	File Summarizer quiz preferences
<code>reviewers-sort-by</code>	localStorage	Sort preference for reviewer board
<code>notepad-imported-to-cloud</code>	localStorage	Whether user has imported local notes to cloud

How Event Listeners Work

An event listener waits for something to happen (a click, a keypress, a scroll) and then runs a function.

```

// Listen for the Enter key in the chat input
chatInput.addEventListener('keydown', function(event) {
  if (event.key === 'Enter') {
    sendMessage();
  }
});

```

How API Calls Work (fetch)

```

// Example: Login request
const response = await fetch('/api/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ username: 'Student1', password: 'mypassword' })
});

const data = await response.json();
// data.token = the JWT token

```

`fetch()` sends HTTP requests from the browser to the server. `await` means "wait for the response before continuing."

How Errors Are Handled

The app uses `try/catch` blocks:

```
try {
  const data = await fetch('/api/something').then(r => r.json());
  // use data
} catch (error) {
  customAlert('Something went wrong: ' + error.message);
}
```

6. Variables and Purposes

Auth / Login Variables

Variable	File	Type	Purpose
<code>SUPABASE_URL</code>	<code>script.js</code>	Global string	The URL of the Supabase project
<code>SUPABASE_KEY</code>	<code>script.js</code>	Global string	The anon (public) key for Supabase
<code>sb</code>	<code>script.js</code>	Global object	The Supabase client instance (used to query the database)
<code>classAppToken</code>	<code>localStorage</code>	String	The JWT token for the logged-in user
<code>classAppUser</code>	<code>localStorage</code>	String	The username of the logged-in user
<code>JWT_SECRET</code>	<code>server.js</code> (.env)	String	Secret key used to sign JWTs on the server (NEVER on frontend)
<code>ADMIN_USERNAME</code>	<code>server.js</code> (.env)	String	Admin account username
<code>ADMIN_PASSWORD</code>	<code>server.js</code> (.env)	String	Admin password (stored as bcrypt hash in .env)

Supabase / Database Variables

Variable	File	Type	Purpose
<code>SUPABASE_URL</code>	<code>.env</code>	String	Cloud database URL
<code>SUPABASE_ANON_KEY</code>	<code>.env</code>	String	Public read key
<code>SUPABASE_SERVICE_KEY</code>	<code>.env</code>	String	Full-access key (backend only!)
<code>sb</code>	<code>script.js</code>	Object	Frontend Supabase client

Storage Variables

Variable	File	Type	Purpose
<code>R2_ENDPOINT</code>	<code>.env</code>	String	Cloudflare R2 base URL
<code>R2_ACCESS_KEY</code>	<code>.env</code>	String	R2 authentication key
<code>R2_SECRET_KEY</code>	<code>.env</code>	String	R2 authentication secret

Variable	File	Type	Purpose
<code>R2_BUCKET</code>	<code>.env</code>	String	Name of the R2 storage bucket
<code>r2Client</code>	<code>server.js</code>	Object	AWS S3-compatible client for R2

AI Service Variables

Variable	File	Type	Purpose
<code>GEMINI_API_KEY</code>	<code>.env</code>	String	Google Gemini AI key (backend only)
<code>GROQ_API_KEY</code>	<code>.env</code>	String	Groq AI key (backup AI)
<code>YOUTUBE_API_KEY</code>	<code>.env</code>	String	YouTube Data API key for music search

PWA / Cache Variables

Variable	File	Type	Purpose
<code>CACHE_VERSION</code>	<code>sw.js</code>	String	Current cache identifier (e.g., <code>v1.5.84-20260511</code>)
<code>ASSETS</code>	<code>sw.js</code>	Array	List of all files to cache for offline use

Page / UI State Variables

Variable	File	Type	Purpose
<code>currentPage</code>	<code>script.js</code> / <code>shell-controls.js</code>	String	Tracks which page is currently visible
<code>currentUser</code>	<code>script.js</code>	Object	Current user's profile data
<code>isAdmin</code>	<code>script.js</code>	Boolean	Whether current user is an admin
<code>CLASS_APP_VERSION</code>	<code>changelog.js</code>	String	Current app version string
<code>CLASS_APP_CHANGELOG</code>	<code>changelog.js</code>	Array	All version entries with descriptions

File Summarizer Variables

Variable	File	Type	Purpose
<code>selectedFile</code>	<code>file-summarizer.js</code>	File	The file chosen by the user
<code>extractedText</code>	<code>file-summarizer.js</code>	String	Text extracted from the uploaded file
<code>quizSettings</code>	localStorage	Object	User's preferred quiz type and count

Reviewer Variables

Variable	File	Type	Purpose
<code>sharedReviewers</code>	<code>reviewers.js</code>	Array	All reviewer cards loaded from Supabase
<code>filtered</code>	<code>reviewers.js</code>	Array	Filtered/sorted subset of reviewers
<code>voteCounts</code>	<code>reviewers.js</code>	Object	Vote tallies per reviewer

Push Notification Variables

Variable	File	Type	Purpose
<code>VAPID_PUBLIC_KEY</code>	<code>.env</code>	String	Public key for push notifications
<code>VAPID_PRIVATE_KEY</code>	<code>.env</code>	String	Private key (server only)

7. Functions and Purposes

Authentication Functions

Function	File	What It Does	Possible Risks
<code>initSupabase()</code>	<code>loading-components.js</code>	Fetches <code>/api/config</code> , creates the Supabase client	If <code>/api/config</code> fails, the whole app can't connect to the database
<code>waitForSupabaseClient()</code>	<code>loading-components.js</code>	Retries Supabase init up to several times with delays	Will show error if Supabase never connects
<code>setServerAuthToken(token)</code>	<code>script.js</code>	Saves JWT to <code>localStorage.classAppToken</code>	Token persists after browser close (security tradeoff)
<code>getServerAuthToken()</code>	<code>script.js</code>	Reads JWT from <code>localStorage.classAppToken</code>	Returns <code>null</code> if not logged in
<code>getAuthHeaders()</code>	<code>script.js</code>	Returns headers object with <code>Authorization: Bearer <token></code>	—
<code>authFetch(url, options)</code>	<code>script.js</code>	<code>fetch()</code> wrapper that automatically includes auth headers	—
<code>ensureSupabaseReady()</code>	<code>script.js</code>	Checks if <code>sb</code> is set; shows alert if not	Without this, database calls would silently fail

Page Navigation Functions

Function	File	What It Does	Parameters	Dependencies
<code>goToPage(pageName)</code>	<code>shell-controls.js</code>	Switches the visible page; cleans up old page, initiates new one	<code>pageName</code> (string)	All page modules
<code>toggleMenu()</code>	<code>shell-controls.js</code>	Opens or closes the sidebar	none	Sidebar HTML element
<code>closeMenu()</code>	<code>shell-controls.js</code>	Closes sidebar if open	none	<code>toggleMenu()</code>
<code>runSafeUiAction(fn, label)</code>	<code>shell-controls.js</code>	Wraps a UI action in try/catch to prevent one error crashing everything	<code>fn</code> (function), <code>label</code> (string)	—

Modal Functions

Function	File	What It Does
<code>customAlert(message)</code>	<code>script.js</code>	Shows a styled popup with one OK button (replaces browser <code>alert()</code>)
<code>customConfirm(message, onConfirm, onCancel)</code>	<code>script.js</code>	Shows a styled popup with OK and Cancel buttons
<code>customPrompt(message, defaultValue, onSubmit)</code>	<code>script.js</code>	Shows a styled popup with a text input field
<code>showToast(message, type, duration)</code>	<code>script.js</code>	Shows a small floating notification in the corner
<code>closeOverlayElement(el)</code>	<code>script.js</code>	Hides a modal overlay element

File System Functions

Function	File	What It Does
<code>fetchAndRenderFolders()</code>	<code>script.js</code>	Queries Supabase <code>folders</code> table, builds folder card HTML
<code>fetchAndRenderFiles(folderId)</code>	<code>script.js</code>	Queries Supabase <code>files</code> table for a folder, renders file list
<code>fetchAndRenderSubFolders(parentId)</code>	<code>script.js</code>	Gets subfolders inside a folder
<code>openFolderExplorer(folderId)</code>	<code>script.js</code>	Opens the folder browser modal
<code>createFolderAPI(name, type)</code>	<code>script.js</code>	Calls <code>/api/folders</code> to create a new folder
<code>deleteFolderAPI(folderId)</code>	<code>script.js</code>	Calls <code>/api/folders/:id</code> DELETE to remove a folder
<code>uploadFileToFolderAPI(folderId, file)</code>	<code>script.js</code>	Uploads a file to R2 via <code>/api/upload</code>
<code>deleteFileAPI(fileId)</code>	<code>script.js</code>	Deletes a file from database and R2

UI Helper Functions

Function	File	What It Does
<code>escapeHTML(str)</code>	<code>script.js</code>	Converts <code><</code> <code>></code> <code>&</code> to safe HTML codes (prevents XSS)
<code>escapeJS(str)</code>	<code>script.js</code>	Escapes special characters for JavaScript strings
<code>formatFileSize(bytes)</code>	<code>script.js</code>	Converts bytes to readable format: "1.2 MB"
<code>isSummarizableFileName(name)</code>	<code>script.js</code>	Returns true if file is PDF, DOC, DOCX, or PPTX
<code>normalizeStoredFileUrl(path)</code>	<code>script.js</code>	Converts stored file paths to full usable URLs
<code>buildSubjectCards(subjects, container)</code>	<code>script.js</code>	Builds subject card HTML for My Classes pages
<code>updateFooterYear()</code>	<code>script.js</code>	Sets the footer copyright year to current year
<code>createInlineLoader(text)</code>	<code>script.js</code>	Returns a spinner HTML string

AI / File Summarizer Functions

Function	File	What It Does
<code>tryWithFallback(provider, messages)</code>	<code>ai-service.js</code>	Calls Gemini → if quota error → calls Groq → if fails → uses local extractor
<code>callGemini(messages)</code>	<code>ai-service.js</code>	Sends messages to <code>/api/gemini</code> and returns response text
<code>callGroq(messages)</code>	<code>ai-service.js</code>	Sends messages to <code>/api/groq</code> and returns response text

Server-Side Functions (server.js)

Function	What It Does
<code>hashPassword(password)</code>	Hashes a password using bcrypt with 12 rounds
<code>verifyPassword(input, hash)</code>	Checks if a plain password matches a bcrypt or SHA-256 hash
<code>issueToken(user)</code>	Creates a 7-day JWT for a logged-in user
<code>fetchSupabaseProfile(username)</code>	Gets a user's profile from Supabase
<code>createSupabaseProfile(data)</code>	Creates a new user profile
<code>requireAuth</code>	Middleware: blocks request if no valid JWT
<code>requireAdmin</code>	Middleware: blocks request if user is not admin
<code>requireSelf(field)</code>	Middleware: blocks request if user is not the resource owner or admin

Session Management Functions

Function	File	What It Does
<code>sessionManager.init()</code>	<code>session-manager.js</code>	Starts token auto-refresh, idle timeout, and multi-tab sync
<code>sessionManager.resetIdleTimer()</code>	<code>session-manager.js</code>	Resets the 30-minute idle countdown
<code>sessionManager.destroy()</code>	<code>session-manager.js</code>	Clears all session timers

Social / Embed Functions

Function	File	What It Does
<code>openSocialPage(pageUrl)</code>	<code>social.js</code>	Opens a Facebook page in an embed iframe
<code>closeSocialPage()</code>	<code>social.js</code>	Hides the embedded page view

8. Authentication Explanation

How Login Works

1. Student fills in username and password on the login screen
2. JavaScript sends `POST /api/login` with the username and password

3. **Server checks the password** using bcrypt (more on this below)
4. If **correct**, the server creates a **JWT token** (a digital ID card)
5. **JavaScript saves the token** in `localStorage` as `classAppToken`
6. **Every future request** includes `Authorization: Bearer <token>` in headers
7. **Server validates** the token on every protected route

How Registration Works

1. Student fills in desired username and password
2. JavaScript sends `POST /api/register`
3. Server hashes the password with **bcrypt** (12 rounds)
4. Server creates the profile in Supabase database
5. Server returns a JWT token → user is immediately logged in

How Logout Works

The app clears the token from localStorage:

```
localStorage.removeItem('classAppToken');
localStorage.removeItem('classAppUser');
```

Without the token, the user is treated as logged out.

How Sessions Work

- Sessions are stored in **localStorage** (stays even after browser is closed)
- The token expires after **7 days**
- `session-manager.js` auto-refreshes the token every 30 minutes
- If the user is idle for 25 minutes, a warning banner appears
- After 30 minutes of idle, the app logs them out automatically
- If the same account logs out in another browser tab, the current tab detects it and logs out too

How User Profiles are Detected

After login, the app calls `/api/session` to verify the token and get user info:

```
const res = await authFetch('/api/session');
const { username, isAdmin } = await res.json();
```

How Admin Checking Works

The server checks if a username is in the `admins` table in Supabase:

```
SELECT 1 FROM public.admins WHERE username = $1
```

If found, `isAdmin = true`. Admins can see extra UI elements and access admin-only API routes.

Authentication Jargon Explained

Term	Simple Explanation
Authentication	Proving who you are (logging in)

Term	Simple Explanation
Authorization	Proving you are allowed to do something
Session	A period of being logged in
Token	A digital "pass" that proves you are logged in
JWT	JSON Web Token — a compact, signed string that contains user info
User ID / UUID	A unique ID number assigned to each user
Role	A label like "student" or "admin" that determines permissions
Admin	A user with extra permissions to manage the app
Protected route	An API endpoint or page that requires login
Client-side auth	Auth logic running in the browser (JavaScript)
Server-side auth	Auth logic running on the server (Node.js)
Supabase Auth	Supabase's built-in authentication system (partially used here)

9. Hash, Salt, Password, and Security

What is a Hash?

A **hash** is a one-way transformation. You feed a password in, and you get a long scrambled string out. You cannot reverse it to get the original password back.

Example:

```

Password: "mypassword123"
Hash:    "$2b$12$LX9R8kP7mN2qF5tH0wE1b0..."

```

The same password always produces a different hash (because of the salt — explained below), which means even if two students have the same password, their hashes look completely different.

What is a Salt?

A **salt** is a random string added to a password before hashing. This prevents "rainbow table attacks" (where an attacker has a giant list of pre-computed hashes).

```

Password + Salt → Hashed Result
"mypassword123" + "random_chars_xyz" → "$2b$12$LX9R8kP7..."

```

In CLASS-APP, bcrypt automatically generates and stores the salt **inside the hash string**. You don't need to handle it separately.

Why Passwords Must NOT Be Stored as Plain Text

If the database is stolen and passwords are stored as plain text:

- Attacker sees `"mypassword123"` directly
- They can log in as any user immediately
- Users who reuse passwords on other sites are also compromised

With hashing, an attacker only sees scrambled strings they cannot reverse.

How Password Hashing Works in CLASS-APP

```
// When registering (server.js):
const hash = await bcrypt.hash(password, 12);
// 12 = "cost factor" – higher = slower = harder to brute-force

// When logging in (server.js):
const match = await bcrypt.compare(enteredPassword, storedHash);
if (match) {
  // Login successful
}
```

Legacy SHA-256 to bcrypt Upgrade

The app previously used SHA-256 (weaker) for passwords. It now auto-upgrades old accounts:

```
function shouldUpgradePasswordHash(hash) {
  return !hash.startsWith('$2b$'); // $2b$ = bcrypt prefix
}
// If SHA-256 hash detected: re-hash with bcrypt on next successful login
```

Does CLASS-APP Handle Password Hashing Directly?

Yes — the backend (`server.js`) uses `bcryptjs` to hash and verify passwords. Supabase Auth is NOT used for the main login flow. The app has its own authentication system backed by Supabase as the profile database only.

Security Concepts Explained

Concept	Simple Explanation	In This App
Hash	One-way scramble of a password	bcrypt in server.js
Salt	Random extra data mixed in before hashing	Built into bcrypt
Encryption	Two-way scramble (can decrypt with key)	Not used for passwords
Decryption	Reversing encryption	Not applicable to passwords
API key	A secret code that allows access to an API	Gemini, YouTube, Groq keys
Public key	A key that can be shared safely	VAPID public key for push
Secret key	A key that MUST stay private	JWT_SECRET, R2 keys
Environment variable	A secret value stored outside the code	<code>.env</code> file
Access token	Short-lived proof of identity	JWT (7 days)
Refresh token	Used to get a new access token	Session auto-refresh
RLS	Row Level Security — database enforces who can read/write each row	All Supabase tables
CORS	Cross-Origin Resource Sharing — controls which domains can call the API	Configured in server.js
XSS	Cross-Site Scripting — attacker injects malicious JS into the page	Prevented by <code>escapeHTML()</code>
CSRF	Cross-Site Request Forgery — attacker tricks user into sending a bad request	Mitigated by JWT in headers
Input validation	Checking that user input is safe and correct	form-validation.js

Concept	Simple Explanation	In This App
File validation	Checking that uploaded files are allowed types/sizes	file-summarizer.js, server.js
Rate limiting	Blocking too many requests in a short time	express-rate-limit in server.js
Secure backend route	An API endpoint protected by auth middleware	Most POST/DELETE routes
Edge Function	Code that runs on a cloud server near users	check-alarms in Supabase
Service role key	A Supabase key that bypasses RLS (NEVER expose to browser)	server.js only
Anon key	A Supabase key for public/authenticated reads	Sent to browser via /api/config

What Should NEVER Be in Frontend Code

Secret	Risk if Exposed
<code>JWT_SECRET</code>	Anyone could forge tokens and pretend to be admin
<code>SUPABASE_SERVICE_KEY</code>	Anyone could read/write all database rows, bypassing RLS
<code>GEMINI_API_KEY</code>	Anyone could rack up huge AI costs on your account
<code>R2_SECRET_KEY</code>	Anyone could access or delete all stored files
<code>GROQ_API_KEY</code>	Anyone could abuse AI credits

CLASS-APP routes all AI and storage calls through `server.js` so these keys stay server-side only. The browser only receives the Supabase **anon** key (safe to expose) via `/api/config`.

10. Database and Supabase Explanation

What is Supabase?

Supabase is a service that provides:

1. A **PostgreSQL database** (organize data in tables, like Excel sheets)
2. **Real-time subscriptions** (get notified when data changes)
3. **Authentication** (Supabase has its own auth, but this app uses custom JWT)
4. **Storage** (file hosting — though this app mostly uses Cloudflare R2)
5. **Edge Functions** (TypeScript code that runs on Supabase's servers)

Database Tables

Based on the 28 migration files, here are the key tables:

Table	Purpose	Key Columns
<code>profiles</code>	User accounts	<code>username</code> , <code>display_name</code> , <code>avatar_url</code> , <code>year_level</code> , <code>password_hash</code>
<code>folders</code>	File organization folders	<code>id</code> , <code>name</code> , <code>owner</code> , <code>type</code> (public/profile/etc), <code>permissions</code>
<code>files</code>	Files stored in folders	<code>id</code> , <code>folder_id</code> , <code>name</code> , <code>url</code> , <code>uploaded_by</code> , <code>size</code>

Table	Purpose	Key Columns
<code>shared_ai_outputs</code>	AI summaries shared publicly	<code>id</code> , <code>username</code> , <code>title</code> , <code>content</code> , <code>created_at</code>
<code>shared_announcements</code>	Admin announcements	<code>id</code> , <code>title</code> , <code>body</code> , <code>is_active</code> , <code>expires_at</code>
<code>app_updates</code>	Software update entries	<code>id</code> , <code>version</code> , <code>title</code> , <code>content</code> , <code>released_at</code>
<code>reviewers</code>	Shared study reviewer cards	<code>id</code> , <code>username</code> , <code>title</code> , <code>content</code> , <code>subject</code> , <code>votes</code>
<code>reviewer_votes</code>	Who voted on which reviewer	<code>id</code> , <code>reviewer_id</code> , <code>username</code> , <code>vote_type</code>
<code>user_notes</code>	Personal notepad notes	<code>id</code> , <code>username</code> , <code>title</code> , <code>content</code> , <code>tags</code> , <code>updated_at</code>
<code>calendar_notes</code>	Calendar sticky notes	<code>id</code> , <code>username</code> , <code>date</code> , <code>note</code> , <code>color</code>
<code>messages</code>	Chat messages	<code>id</code> , <code>room</code> , <code>sender</code> , <code>content</code> , <code>created_at</code>
<code>admins</code>	Admin user list	<code>username</code> , <code>granted_at</code> , <code>granted_by</code> , <code>reason</code>
<code>candy_scores</code>	Candy Match game scores	<code>id</code> , <code>username</code> , <code>score</code> , <code>level</code>
<code>lobby_scores</code>	Lobby platformer scores	<code>id</code> , <code>username</code> , <code>stars_collected</code>
<code>tetris_leaderboard</code>	Tetris scores	<code>id</code> , <code>username</code> , <code>score</code> , <code>lines</code>
<code>code_lab_completions</code>	Coding lesson completions	<code>id</code> , <code>username</code> , <code>lesson_id</code> , <code>completed_at</code>
<code>push_subscriptions</code>	Web push subscription endpoints	<code>id</code> , <code>username</code> , <code>subscription_json</code>
<code>operation_audit_log</code>	Admin action log	<code>id</code> , <code>actor</code> , <code>action</code> , <code>target</code> , <code>created_at</code>
<code>summary_history</code>	File summarizer history	<code>id</code> , <code>username</code> , <code>filename</code> , <code>summary_type</code> , <code>created_at</code>
<code>quiz_history</code>	Quiz attempt records	<code>id</code> , <code>username</code> , <code>questions</code> , <code>score</code> , <code>created_at</code>
<code>subjects</code>	Subject/course definitions	<code>id</code> , <code>name</code> , <code>year_level</code> , <code>semester</code> , <code>teacher</code>

Row Level Security (RLS)

RLS is the database's way of enforcing who can see and modify what. It's like a bouncer at the database level.

```
-- Example: Only the folder owner or admin can delete a folder
CREATE POLICY "owner_or_admin_can_delete"
ON folders FOR DELETE
USING (owner = current_setting('app.username') OR class_app_is_admin());
```

Key RLS Helper Functions:

- `class_app_is_admin()` — checks if current user is in the `admins` table
- `class_app_can_view_folder(folder_id)` — checks folder permissions
- `class_app_can_edit_folder(folder_id)` — checks edit access

The frontend sets the current username in a request header (`x-class-username`), and the database reads it via `current_setting('app.username')` to enforce policies.

How Data Flows

Reading data (SELECT):

```
// frontend script.js:
const { data, error } = await sb
  .from('folders')
  .select('id, name, type, owner')
  .order('created_at', { ascending: false });
```

Creating data (INSERT):

```
const { data, error } = await sb
  .from('user_notes')
  .insert({ username: currentUser, title: 'My Note', content: 'Hello' });
```

Updating data (UPDATE):

```
const { error } = await sb
  .from('user_notes')
  .update({ content: 'Updated content' })
  .eq('id', noteId);
```

Deleting data (DELETE):

```
const { error } = await sb
  .from('user_notes')
  .delete()
  .eq('id', noteId);
```

Database Jargon Explained

Term	Simple Explanation
Database	A structured collection of data (like many Excel sheets organized together)
Table	One "sheet" with rows and columns
Row	One record (one user, one file, one message)
Column	One property of a record (name, date, size)
Primary key	A unique ID for each row (<code>id</code>)
Foreign key	A column that points to another table's primary key (<code>folder_id</code> in files points to <code>folders.id</code>)
Query	A request to read or change data
INSERT	Add a new row
SELECT	Read rows
UPDATE	Change existing rows
DELETE	Remove rows
Realtime	Supabase feature that pushes data changes to browsers instantly
Storage bucket	A folder in the cloud for storing files
Public URL	A file URL anyone can access without authentication
Signed URL	A temporary URL with a built-in expiry, only valid for limited time

11. Storage and File Uploads

Where Files Are Stored

CLASS-APP uses **Cloudflare R2** for file storage. R2 is compatible with Amazon S3's API and is used because it's cheaper (no bandwidth fees).

Files are stored in the `R2_BUCKET` bucket. Subfolders inside:

- `uploads/images/` — uploaded images
- `uploads/videos/` — uploaded videos
- `uploads/audio/` — audio files
- `uploads/documents/` — PDFs, DOCX, PPTX

How File Upload Works (Step by Step)

1. User selects a file using `<input type="file">`
2. JavaScript creates a `FormData` object and appends the file
3. JavaScript sends `POST /api/upload` with auth headers and the `FormData`
4. Server receives the file via **multer** (a file upload middleware)
5. Server validates file type and size (max 50MB for video, 8MB for documents in summarizer)
6. Server uploads the file to Cloudflare R2 using the AWS S3 SDK
7. Server saves the file record (name, URL, folder_id) to Supabase `files` table
8. Frontend receives the file URL and shows it in the UI

File Summarizer Upload Flow

1. User uploads PDF, DOC, DOCX, or PPTX (max 8MB)
2. Server extracts text:
 - **PDF** → `pdf-parse` library reads text
 - **DOCX/DOC** → `mammoth` library extracts text
 - **PPTX** → `adm-zip` opens ZIP, parses slide XML tags
3. Extracted text (max 50,000 characters) is sent to AI:
 - Gemini 1.5 Flash (primary)
 - Groq (if Gemini quota exceeded)
 - Local sentence extractor (if both fail)
4. AI generates: study notes, detailed summary, key concepts, terms, or quiz

File Validation

What is validated:

- File type (extension): only .pdf, .doc, .docx, .pptx for summarizer
- File size: max 8MB for summarizer, 50MB for video uploads
- Text content: at least 1 character of extractable text (rejects image-only PDFs)

What is NOT validated / risks:

- MIME type not deeply verified (extension check only)
- No virus/malware scanning

- No image content moderation

File Security — The `/uploads/*` Route

```
// server.js: Serving files from R2
app.get('/uploads/*', async (req, res) => {
  // Fetch from R2 using the S3 SDK
  // Set safe Content-Type (never text/html or text/svg)
  // Set X-Content-Type-Options: nosniff header
  // For certain types, force download (attachment) instead of display (inline)
});
```

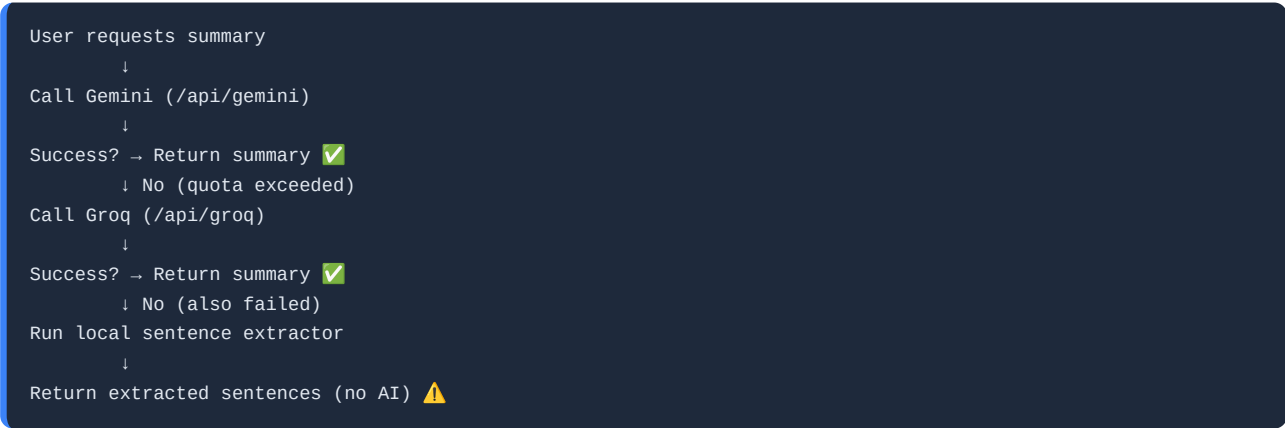
The `X-Content-Type-Options: nosniff` header prevents browsers from guessing the file type, which could allow an attacker to serve a JavaScript file disguised as an image.

12. AI Integrations

AI Services Used

Service	Model	Purpose	Where Key Lives
Google Gemini	gemini-1.5-flash	File summaries, quiz generation, AI chat	<code>server.js</code> (env)
Groq	llama-3 (or similar)	Fallback when Gemini quota is exceeded	<code>server.js</code> (env)
Local extractor	(no model)	Emergency fallback — extracts ~30% of sentences from text	<code>ai-service.js</code>

Three-Tier Fallback System



File Summarizer AI Flow

When a user uploads a PDF and requests a "detailed summary":

1. Frontend: file → `POST /api/summarize-file`
2. Backend: extract text → build AI prompt → call Gemini
3. Prompt example: `"Summarize the following document as study notes for a student: [extracted text]"`
4. Backend: return AI response
5. Frontend: display summary, enable quiz button

Why API Keys Must Be Backend-Only

If `GEMINI_API_KEY` were in the frontend JavaScript (visible to anyone who views page source):

- Anyone could make unlimited AI requests on your Google account
- Costs could reach thousands of dollars
- The key could be revoked, breaking the app for everyone

CLASS-APP keeps all AI keys in `.env` and only exposes them through controlled server routes with **rate limiting** (`aiLimiter`: max 10 requests per 60 seconds per IP).

YouTube Music Search

```
// server.js
app.get('/api/yt-search', requireAuth, async (req, res) => {
  const results = await fetch(
    `https://www.googleapis.com/youtube/v3/search?key=${YOUTUBE_API_KEY}&q=${query}`
  );
  res.json(results);
});
```

The YouTube API key is called server-side. The browser never sees the key — it only sends a search query to `/api/yt-search`.

Three fallback methods exist for YouTube:

1. Official YouTube Data API v3 (requires key, quota-limited)
2. Piped API (no auth needed, open source proxy)
3. InnerTube API (simulates a mobile YouTube client, no key needed)

13. PWA, Cache, and Service Worker

What Makes CLASS-APP a PWA?

1. **manifest.json** — tells browsers the app can be installed
2. **sw.js** — service worker that caches files for offline use
3. **HTTPS** — required for service workers
4. **Icons** — proper 192×192 and 512×512 PNG icons

What manifest.json Does

```
{
  "name": "My School Portfolio",
  "short_name": "School Portfolio",
  "display": "standalone",           - opens like a native app (no browser bar)
  "start_url": "/",
  "background_color": "#020617",    - splash screen color
  "theme_color": "#0b1b3d",         - phone status bar color
  "icons": [...]
}
```

When a user visits the app and Chrome/Safari detects a valid manifest + service worker, it may show an "Add to Home Screen" prompt.

What sw.js Does

The service worker is a special JavaScript file that runs in the background (not on the main page). It intercepts all network requests and can serve files from cache.

Key parts of sw.js:

1. Installation — caching files:

```
self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_VERSION).then((cache) => {
      return cache.addAll(ASSETS); // Download and cache all 92 files
    })
  );
});
```

2. Activation — cleanup:

```
self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((keys) => {
      return Promise.all(
        keys.filter(key => key !== CACHE_VERSION).map(key => caches.delete(key))
      );
    })
  );
});
```

When a new version is deployed, old caches are deleted so users get fresh files.

3. Fetch — serving from cache:

```
self.addEventListener('fetch', (event) => {
  // Never cache /api/ calls (always fresh data)
  if (event.request.url.includes('/api/')) return;

  // For everything else: try network first, fallback to cache
  event.respondWith(
    fetch(event.request).catch(() => caches.match(event.request))
  );
});
```

What Cache Version Means

```
const CACHE_VERSION = "v1.5.84-20260511-tetris-restored";
```

Every time the app is updated, this string changes. The service worker detects the new version, deletes the old cache, and downloads fresh files.

Why Cache Must Be Bumped After Updates

If you change `style.css` but don't bump the cache version, users will:

1. Download the new service worker
2. But the service worker still has the OLD CSS version in cache
3. Users see old styles until they manually clear browser cache

This is why every change requires updating `?v=N` in `index.html` AND `sw.js`, and changing `CACHE_VERSION`.

The app has `scripts/cache-bust.js` that runs automatically before deployment (`npm run prestart`) to stamp the cache version with the current date.

How to Debug Cache Issues

1. Open Chrome DevTools → Application → Service Workers
2. Click "Update" to force-update the service worker
3. Click "Unregister" to completely clear the service worker
4. Then hard refresh: `Ctrl+Shift+R` (or `Cmd+Shift+R` on Mac)

Push Notifications

The app supports Web Push notifications for alarms. This uses:

- **VAPID keys** — authentication keys for push services
- `web-push` npm package — sends notifications from the server
- `supabase/functions/check-alarms/index.ts` — Edge Function that checks if alarms are due and sends push

PWA Jargon Explained

Term	Simple Explanation
PWA	A website that acts like a native app
Service worker	Background script that controls caching and push notifications
Manifest	JSON file that tells browsers how to install the app
Cache	Saved copies of files stored locally for fast/offline loading
Cache busting	Forcing browsers to download new file versions
Offline mode	App works (partially) without internet using cached files
Install prompt	Browser popup asking "Add to home screen?"
Fetch event	The service worker intercepts every file/API request
Push notification	Server sends a message that appears even when app is closed
Background sync	Queuing offline actions to send when internet returns

14. Page-by-Page Feature Explanation

Homepage

- **Purpose:** Landing page after login, shows welcome message and key feature cards
- **Files:** `index.html` (home-page div), `script.js` (buildSubjectCards)
- **HTML:** `<div id="home-page" class="page active">`
- **What happens:** Shows personalized greeting, quick-access cards, live clock
- **Auth:** User must be logged in; shows username

Lobby

- **Purpose:** A multiplayer platform game where students control characters that collect stars

- **Files:** `features/lobby/lobby.js`, `server.js` (Socket.IO events: `lobby:join`, `lobby:move`, `lobby:collect`)
- **Real-time:** Uses Socket.IO for player movement synchronization
- **Database:** `lobby_scores` table in Supabase

Announcements

- **Purpose:** Admin-posted announcements visible to all students
- **Files:** `script.js`, `supabase/migrations/006_activity_log_and_subject_announcements.sql`
- **Database:** `shared_announcements` table (viewable by all if `active=true`)
- **Auth:** Only admins can post; anyone logged in can view

My Classes (Year 1 – 4, Semester 1 & 2)

- **Purpose:** Browse class files organized by year and semester
- **Files:** `features/academics/academics.js`, `script.js` (buildSubjectCards)
- **Subjects:** Hard-coded in `script.js` as arrays (`firstSem[]`, `secondSem[]`, `y2firstSem[]`, etc.)
- **Year 3 and 4:** Currently empty (no subjects defined yet)
- **Folders:** Each subject can have folders; uses the folder/file system below

File Manager (Folders System)

- **Purpose:** Upload, organize, and download files in folders
- **Files:** `script.js` (fetchAndRenderFolders, fetchAndRenderFiles, all folder/file API functions)
- **Database:** `folders` table, `files` table
- **Storage:** Cloudflare R2
- **Auth:** Folder visibility controlled by RLS (`class_app_can_view_folder`)
- **Features:** Create/rename/delete folders, upload files, share folders, copy/move files

File Summarizer

- **Purpose:** Upload a PDF/DOC/DOCX/PPTX, get an AI-generated summary or quiz
- **Files:** `features/file-summarizer/file-summarizer.js`, `features/file-summarizer/file-summarizer.css`
- **Backend:** `POST /api/summarize-file`, `POST /api/quiz`
- **AI:** Gemini 1.5 Flash → Groq → local extractor
- **Summary types:** Short, Detailed, Key Concepts, Terms & Definitions, Quiz
- **Quiz types:** Identification, Multiple Choice, Short Answer, Mixed
- **State:** Quiz settings saved in `localStorage` (`fs-quiz-settings`)

Shared Reviewer

- **Purpose:** Students post and vote on study reviewer cards
- **Files:** `features/reviewers/reviewers.js`, `features/reviewers/reviewers.css`
- **Database:** `reviewers` table, `reviewer_votes` table
- **Features:** Create, view, vote up/down, search, filter by subject, share from Notepad
- **Auth:** Anyone can view; logged-in users can vote; only creator or admin can delete

AI Assistants

- **Purpose:** Chat directly with AI models (Gemini or Groq)
- **Files:** `features/ai/ai.js`, `features/ai/ai-config.js`

- **Backend:** `POST /api/gemini` , `POST /api/groq`
- **Rate limit:** 10 requests per 60 seconds

Coding Lessons

- **Purpose:** Interactive coding lessons with code editor and execution
- **Files:** `coding-educational/coding-educational.js` , `coding-educational/coding-educational-data.js` (133KB of lesson content)
- **Features:** HTML/CSS/JS lessons, Java lessons, code execution (Java runs server-side via subprocess), progress tracking
- **Database:** `code_lab_completions` , `subjects` table

Chat

- **Purpose:** Real-time group and private messaging
- **Files:** `features/chat/chat.js` , `server.js` (Socket.IO)
- **Real-time:** Socket.IO rooms (`joinChat` , `sendMessage`)
- **Database:** `messages` table in Supabase (history)
- **Auth:** Must be identified via Socket.IO `identify` event with JWT

User Directory

- **Purpose:** See all registered users, who is online
- **Files:** `features/users/users.js`
- **Database:** Supabase `profiles` table (public fields only)
- **Presence:** Real-time online status via Supabase Realtime or Socket.IO presence

Social Media Pages

- **Purpose:** View embedded Facebook pages
- **Files:** `features/social/social.js`
- **How:** Creates an `<iframe>` with Facebook's embed URL
- **Risk:** Facebook may block iframe embeds; the app shows a user-friendly error after 8 seconds if loading fails

Music Hub

- **Purpose:** Search and play YouTube music directly in the app
- **Files:** `features/music/music.js`
- **Backend:** `/api/yt-search` (official YouTube API), `/api/piped-search` (Piped proxy), `/api/yt-scrape` (InnerTube)
- **Fallback:** Three search methods in case one fails

Games

The app has five full games:

Pokémon Battle (`features/pokemon/pokemon.js` — 161KB)

A turn-based Pokémon battle game with battle mechanics, type effectiveness, moves, and health bars.

Battle Royale (`features/royale/royale.js` — 151KB)

A multiplayer battle arena game with real-time movement, combat, and leaderboards.

Candy Match (`features/candy/candy.js` — 62.8KB)

A match-3 style candy puzzle game (similar to Candy Crush). Uses `candy_scores` table.

Pac-Man (`features/pacman/pacman.js` — 11.2KB)

Classic Pac-Man game implemented with canvas rendering.

Tetris (`features/tetris/tetris.js` — 21.1KB)

Classic Tetris game with leaderboard stored in `tetris_leaderboard` table.

Lobby Platformer (`features/lobby/`)

A real-time multiplayer platformer where players collect stars. Score stored in `lobby_scores`.

Personal Tools**Alarm Clock (`features/personal-tools/alarm-clock.js` — 24.6KB)**

- 20 different alarm sounds generated with Web Audio API (no audio files needed)
- Push notifications via `check-alarms` Edge Function
- Snooze, dismiss actions

Notepad (`features/personal-tools/notepad.js` — 693 lines)

- Cloud sync to Supabase `user_notes` table
- Offline fallback to `localStorage`
- Tags, undo delete (5-second grace period)
- Share notes directly to Shared Reviewer

Scientific Calculator (`features/personal-tools/calculator.js` — 18.5KB)

- Full scientific calculator with trig functions, logarithms, memory
- Dark themed, glassmorphism design

Personalization (`features/personal-tools/personalization.js` — 13.2KB)

- Background presets from `features/personalization/background-presets.js` (44+ presets)
- Categories: IT Systems, Anime, Games, School, Space, Nature, City, Abstract, Music
- Custom background image upload
- Theme color adjustments

Calendar

- **Purpose:** Personal calendar with sticky notes per date
- **Files:** `features/calendar/calendar.js`
- **Database:** `calendar_notes` table
- **Features:** Click any date to add/edit a sticky note

Software Update Page

- **Purpose:** Show the changelog of all app versions
- **Files:** `features/updates/updates.js`, `features/updates/changelog.js` (137.9KB — very large!)
- **Features:** Version grid, update banner for new versions, update notification badge
- **Data:** `CLASS_APP_CHANGELOG` array in `changelog.js`

Admin Features

- **Diagnostics:** System health, memory usage, R2 status, Java availability
- **User management:** View, edit, delete users (admin only)
- **Admin check:** Verified server-side via `requireAdmin` middleware and `admins` database table

PWA Install Feature

- **How it works:** Browser fires `beforeinstallprompt` event → app captures it → custom "Install App" button appears → user clicks → browser install prompt shown
- **Platform notes:** Works on Chrome/Edge/Android; limited on iOS Safari (must use "Add to Home Screen" manually)

15. Button and Event Listener Map

This table maps key buttons to their JS functions. Buttons marked ⚠️ may be at risk from the `pointerEvents` bug documented in MUST-FIX.

Button	HTML ID/Class	Function Called	File	Connection Method	Risk
Hamburger menu	<code>#hamburger-btn</code>	<code>toggleMenu()</code>	<code>shell-controls.js</code>	<code>onclick="toggleMenu()"</code> (inline)	⚠️ Inline — must be on window
Home nav item	<code>.nav-item[data-page="home"]</code>	<code>goToPage('home')</code>	<code>shell-controls.js</code>	Event delegation on <code>#nav-list</code>	✅
File Summarizer nav	<code>.nav-item[data-page="file-summarizer"]</code>	<code>goToPage('file-summarizer')</code>	<code>shell-controls.js</code>	Event delegation	✅
Upload file btn	<code>#fs-upload-btn</code> or <code>#fs-file-input</code>	<code>handleFileUpload()</code>	<code>file-summarizer.js</code>	<code>addEventListener('change')</code>	✅
Generate Summary	<code>#fs-generate-btn</code>	<code>generateSummary()</code>	<code>file-summarizer.js</code>	<code>addEventListener('click')</code>	✅
Generate Quiz	<code>#fs-quiz-btn</code>	<code>generateQuiz()</code>	<code>file-summarizer.js</code>	<code>addEventListener('click')</code>	✅
Create Folder	<code>#create-folder-btn</code>	<code>createFolderAPI()</code>	<code>script.js</code>	<code>onclick</code> attribute	⚠️ Must be on window
Delete Folder	<code>.folder-delete-btn</code>	<code>deleteFolderAPI(id)</code>	<code>script.js</code>	Event delegation	✅
Vote Up (reviewer)	<code>.vote-up-btn</code>	<code>voteOnReviewer('up', id)</code>	<code>reviewers.js</code>	Event delegation	✅

Button	HTML ID/Class	Function Called	File	Connection Method	Risk
Share Note to Reviewer	#share-to-reviewer-btn	shareToReviewer()	notepad.js	addEventListener('click')	✓
Install App	#install-btn	Shows browser install prompt	script.js	Event listener on beforeinstallprompt	✓
Set Alarm	#set-alarm-btn	addAlarm()	alarm-clock.js	addEventListener('click')	✓
Send Chat	#chat-send-btn	sendMessage()	script.js	Socket.IO after identify	✓
Open Social	Social card	openSocialPage(url)	social.js	onclick or event listener	⚠
Software Update tab	Sidebar nav	goToPage('updates')	shell-controls.js	Event delegation	✓

The Critical pointerEvents Bug

Documented in `MUST-FIX` file: in `script.js` around line 3197:

```
// CURRENT (BROKEN):
page.style.pointerEvents = page.classList.contains('active') ? '' : '';
//                               ↑      ↑
//                               Both branches are empty strings! This does nothing.

// CORRECT:
page.style.pointerEvents = page.classList.contains('active') ? 'auto' : 'none';
```

Impact: Non-active (hidden) pages still receive click events. A button on Page B can accidentally be clicked even when Page A is visible, causing silent bugs.

16. Common Bug Patterns in the App

1. The pointerEvents Ternary Bug (CRITICAL — documented in MUST-FIX)

Location: `script.js` ~line 3197

Symptom: Buttons on hidden pages can still be triggered; weird behavior when navigating

Fix: Change both branches of the ternary to `'auto'` and `'none'` respectively

2. Inline onclick Calling Undefined Function

Symptom: Button appears, hovers correctly (CSS works), but clicking does nothing. No error in console (or a "function not defined" error).

Cause: HTML has `onclick="doSomething()"` but `doSomething` is not on `window.doSomething`.

Example:

```
<button onclick="openFolderExplorer(id)">Open</button>
```

```
// Bug: function is inside a module scope, not on window
function openFolderExplorer(id) { ... }

// Fix: expose it
window.openFolderExplorer = openFolderExplorer;
```

3. Script Loading Order Problems

Symptom: Error like "customAlert is not defined" on page load

Cause: A script tries to call a function from another script that hasn't loaded yet

How to check: Look at script tag order in `index.html` — scripts run top to bottom

4. Old Cache Serving Outdated Code

Symptom: You deployed a fix, but users still see the old version

Cause: Service worker still has old files in cache, and `CACHE_VERSION` wasn't updated

Fix: Bump `CACHE_VERSION` in `sw.js` and bump `?v=N` for changed files in `index.html` AND `sw.js`

5. Duplicate Function Names

Symptom: Only one version of a function runs — unexpected behavior

Cause: Two files define `function uploadFile()` — the second one overwrites the first

How to find: `grep -n "function uploadFile" **/*.js`

6. Overlays Blocking Buttons

Symptom: Button is visible, but clicking it doesn't work — no function is called

Cause: An invisible overlay `div` is on top of the button (z-index or position issue)

How to find: Chrome DevTools → Inspect Element → look for invisible divs with high z-index

7. CSS pointer-events: none on Parent

Symptom: Button inside a container is not clickable even though it has a click handler

Cause: Parent element has `pointer-events: none` which inherits to children

Fix: Add `pointer-events: auto` to the specific button element

8. Supabase Errors Silently Failing

Symptom: Data doesn't save or load; no visible error

Cause: `const { data, error } = await sb.from(...).select(...)` — `error` is never checked

Fix: Always check: `if (error) { console.error(error); customAlert('Database error'); return; }`

9. RLS Permission Errors

Symptom: Supabase returns empty array or 403 error for valid queries

Cause: Row Level Security policy blocks the request

How to debug: Check Supabase dashboard → Table Editor → check RLS policies for the table

10. JavaScript Error Stopping the Rest of the File

Symptom: Everything below a certain line in a JS file stops working

Cause: An uncaught runtime error stops execution

Fix: Use `try/catch` around risky operations; check browser console for the exact error

11. Broken Imports (Modules Not Loading)

Symptom: `window.reviewersModule is undefined`

Cause: The feature JS file wasn't loaded in `index.html` or failed to load

Fix: Check the `<script>` tag exists, check for loading errors in the Network tab

12. Mobile Layout Conflicts

Symptom: Button is visible on desktop, invisible or unclickable on phone

Cause: Fixed-position elements overlapping, 100vh including browser chrome on iOS, safe area not respected

Fix: Use `env(safe-area-inset-*)` CSS variables for iOS; test on actual phone

13. Undefined Variables

Symptom: `Cannot read properties of undefined (reading 'username')`

Cause: Code assumes `currentUser` is loaded but it isn't yet (async timing issue)

Fix: Check if the variable is loaded before using it; use optional chaining `currentUser?.username`

14. Ana Cruz Demo Profile (Hardcoded Data)

Location: `coding-educational/coding-educational.js` (lines 420, 425, 437)

Issue: A hardcoded demo student profile named "Ana Cruz" appears in lesson completion data

Risk: Confuses real data with demo data; should be clearly labeled as mock/demo data

17. Glossary of Jargon

Term	Simple Explanation
HTML	HyperText Markup Language — the skeleton/structure of a webpage
CSS	Cascading Style Sheets — the visual styling (colors, sizes, layout)
JS / JavaScript	Programming language that makes webpages interactive
DOM	Document Object Model — the browser's live representation of the HTML page as objects JavaScript can read and modify
API	Application Programming Interface — a way for two programs to talk to each other; in this app, the browser talks to <code>server.js</code> via API routes
async/await	JavaScript syntax for handling code that takes time (like waiting for a server response) without freezing the page
Promise	An object representing a future value — either resolved (success) or rejected (error)
fetch	A JavaScript function that sends HTTP requests (like going to a URL and getting data back)
JSON	JavaScript Object Notation — a text format for sending structured data (like <code>{"name": "Student", "year": 1}</code>)
Array	An ordered list of values: <code>["item1", "item2", "item3"]</code>
Object	A collection of named values: <code>{ username: "Student1", year: 1 }</code>
Variable	A named container for a value (<code>let score = 100</code>)
Constant	A variable that cannot be reassigned (<code>const MAX_SIZE = 50</code>)

Term	Simple Explanation
Function	A reusable block of code that does a specific task
Parameter	Input values a function accepts (<code>function greet(name) {...}</code>)
Return	The value a function sends back to the caller
Event listener	Code that waits for a specific event (click, keypress) and runs a function
Callback	A function passed as an argument to another function, to be called later
Module	A separate JS file with its own scope; must explicitly export/expose what it wants to share
import/export	JavaScript syntax for sharing code between files (not used here — the app uses <code>window.</code> globals instead)
CDN	Content Delivery Network — a network of fast servers that hosts libraries (like Supabase JS, Socket.IO)
package.json	Lists the project's name, version, start commands, and all npm packages it depends on
Dependency	An npm package the project needs to run
npm	Node Package Manager — installs JavaScript packages/libraries
Build	Compiling/bundling code into production-ready files (this app uses no build step — files run directly)
Deployment	Making the app available on the internet (this app deploys to Render.com)
Render	A cloud hosting platform (like Heroku) where <code>server.js</code> runs
Supabase	A cloud service providing PostgreSQL database, authentication, and real-time subscriptions
Cloudflare R2	Cloud file storage (like a giant hard drive in the cloud) — compatible with Amazon S3 API
Database	A structured system for storing and retrieving data
Authentication	Proving who you are (login)
Authorization	Proving you are allowed to do something
Cache	Saved copies of files that load faster than re-downloading from the server
Service worker	A background script that intercepts network requests to enable caching and push notifications
PWA	Progressive Web App — a website that installs on a phone like an app
Hash	A one-way mathematical transformation of a password into a fixed-length scrambled string
Salt	Random data added to a password before hashing, making each hash unique
Token	A digital "proof" of authentication — usually a long string
JWT	JSON Web Token — a signed token containing user info (like <code>{ username: "Student1", exp: ... }</code>)
RLS	Row Level Security — database rules that control which rows each user can access
Environment variable	A secret value stored outside the code (in <code>.env</code> file) to keep keys secure
Backend	The server-side code that handles data, auth, and business logic (<code>server.js</code>)
Frontend	The browser-side code the user interacts with (<code>index.html</code> , <code>script.js</code> , CSS files)
Edge Function	Code that runs on cloud servers distributed globally (Supabase's <code>check-alarms</code> function)
CORS	Cross-Origin Resource Sharing — browser security that controls which domains can access an API

Term	Simple Explanation
XSS	Cross-Site Scripting — an attack where malicious JavaScript is injected into a page
CSRF	Cross-Site Request Forgery — an attack that tricks a user's browser into sending requests to an API
Validation	Checking that input data is correct (e.g., file must be under 8MB)
Sanitization	Cleaning input data to remove dangerous content (e.g., removing <code><script></code> tags)
Rate limit	A limit on how many requests an IP can make in a given time (prevents abuse)
Storage bucket	A container in cloud storage for files
Public URL	A URL anyone can access without authentication
Signed URL	A temporary URL that expires after a set time
CRUD	Create, Read, Update, Delete — the four basic database operations
Responsive design	A layout that adapts to different screen sizes (phone, tablet, desktop)
Media query	CSS rules that apply only at certain screen widths
z-index	A CSS property controlling stacking order (higher = in front)
localStorage	Browser storage that persists even after the browser is closed (per domain)
sessionStorage	Browser storage that clears when the browser tab is closed
Socket.IO	A library for real-time two-way communication between browser and server
bcrypt	A cryptographic algorithm specifically designed for hashing passwords (used in server.js)
Middleware	Functions in Express that run before route handlers (e.g., <code>requireAuth</code> checks tokens)
Multer	An Express middleware for handling file uploads
Helmet	An Express middleware that sets secure HTTP headers
CSP	Content Security Policy — a security header that restricts what scripts/styles can be loaded
Docker	A platform that packages an app with all its dependencies into a container
Render.com	A cloud hosting service where this app is deployed
PostgreSQL	A powerful open-source relational database (what Supabase runs on)
Migration	A SQL file that incrementally changes the database structure (table creation, policy changes)
Realtime	Supabase feature that pushes database changes to connected clients instantly
VAPID	Voluntary Application Server Identification — keys used to authenticate Web Push notifications
Web Audio API	A browser API that generates audio programmatically (used by the alarm clock)
Canvas API	A browser API for drawing graphics (used by the Pac-Man game, starfield background)

18. Learning Roadmap

This roadmap is designed specifically for you as a CLASS-APP owner learning to understand and maintain your own app.

Level 1: HTML Basics from Your App

What to study:

- What HTML tags mean (`div` , `button` , `input` , `form` , `nav` , `ul` , `li` , `img` , `audio`)
- The difference between `id` and `class`
- How HTML is structured (parent/child elements)

Files to open:

- `index.html` — focus on lines 1 to 200 first
- Look for the `<div id="home-page">` section
- Find the sidebar `<nav id="sidebar">` section

Practice task: Add a new `<p>` tag with a welcome message somewhere inside `#home-page` and see it appear.

Mistakes to avoid:

- Forgetting to close tags (`<div>` must have `</div>`)
- Giving two elements the same `id` (each ID must be unique)

Level 2: CSS and Layout from Your App

What to study:

- Selectors: `#id` , `.class` , `element`
- Flexbox: `display: flex` , `flex-direction` , `justify-content` , `align-items`
- Glassmorphism: `backdrop-filter: blur` , `rgba()` background
- Media queries: how the app changes on smaller screens

Files to open:

- `style.css` — read sections with comments (look for `/* === */` dividers)
- `features/personal-tools/notepad.css` — a smaller, manageable CSS file

Practice task: Change the `.card` background color to a light blue by modifying the `style.css` .

Mistakes to avoid:

- Overriding styles without checking if the class is used globally
- Forgetting that CSS cascades — later rules override earlier ones

Level 3: JavaScript Buttons and DOM

What to study:

- `document.getElementById()` — finding elements
- `element.classList.add/remove/toggle()` — showing/hiding things
- `addEventListener('click', function)` — making buttons work
- How `goToPage()` switches between pages

Files to open:

- `features/logging-in/shell-controls.js` — study `goToPage()` and `toggleMenu()`
- `features/personal-tools/notepad.js` — study how click handlers are attached

Practice task: Find the hamburger button handler in `shell-controls.js` and trace exactly what happens when it's clicked.

Mistakes to avoid:

- Writing `element.addEventListener` before the element exists in the DOM
 - Forgetting `window.functionName = functionName` when using inline `onclick`
-

Level 4: Authentication and User Sessions

What to study:

- How JWT tokens work (sign in → get token → include in every request)
- How `localStorage` stores the token
- What `requireAuth` middleware does in `server.js`
- How `session-manager.js` handles token refresh and idle timeout

Files to open:

- `features/logging-in/loading-components.js` — auth bootstrap
- `features/security/session-manager.js` — session lifecycle
- `server.js` lines 200-400 — auth routes and middleware

Practice task: Manually trace the login flow: from "student clicks Sign In" to "JWT stored in localStorage."

Mistakes to avoid:

- Exposing `JWT_SECRET` anywhere in frontend code
 - Trusting client-side `isAdmin` checks without server validation
-

Level 5: Supabase Database and Storage

What to study:

- How to read from a Supabase table: `sb.from('tableName').select()`
- How to insert data: `sb.from('tableName').insert({...})`
- What RLS policies do and why they matter
- The difference between anon key and service key

Files to open:

- `supabase/migrations/002_tables.sql` — what tables exist
- `supabase/migrations/003_rls_policies.sql` — what the security rules are
- `features/personal-tools/notepad.js` — look at how it syncs notes with Supabase

Practice task: Open the Supabase dashboard, go to Table Editor, and view a few rows in the `user_notes` table.

Mistakes to avoid:

- Using the service key in frontend code (it bypasses all RLS!)
 - Not handling the `error` returned by every Supabase call
-

Level 6: File Uploads and AI Summarization

What to study:

- How `multer` receives uploaded files in `server.js`
- How `pdf-parse` and `mammoth` extract text
- The AI prompt structure sent to Gemini
- How the three-tier fallback works in `ai-service.js`

Files to open:

- `features/file-summarizer/file-summarizer.js` — full file (it's ~1,000 lines)
- `ai-service.js` — the fallback chain
- `server.js` around the `/api/summarize-file` and `/api/quiz` routes

Practice task: Upload a PDF in the app, open Chrome DevTools Network tab, and watch the request to `/api/summarize-file`.

Mistakes to avoid:

- Putting AI API keys in frontend code (anyone can steal them from page source)
 - Forgetting file size and type validation on the server side
-

Level 7: PWA, Service Worker, and Caching

What to study:

- What `manifest.json` does and why each field matters
- How `sw.js` installs, activates, and intercepts fetch events
- What `CACHE_VERSION` is and why it changes
- How to force-update the service worker during development

Files to open:

- `manifest.json` — all fields
- `sw.js` — all 285 lines (it's readable!)
- `scripts/cache-bust.js` — how the version is auto-stamped

Practice task: Change `CACHE_VERSION` in `sw.js` to a new value, reload the app, and watch DevTools → Application → Cache Storage update.

Mistakes to avoid:

- Forgetting to bump `?v=N` in both `index.html` and `sw.js`
 - Forgetting to update `CACHE_VERSION` after any frontend change
-

Level 8: Security and Backend Protection

What to study:

- How bcrypt hashing works and why 12 rounds
- What CSP (Content Security Policy) does in `server.js` Helmet config
- How rate limiting works (`loginLimiter` , `aiLimiter` , `folderFileLimiter`)

- What RLS means and how the policies in migration 003 and 015 protect data
- What environment variables are and how `.env` works

Files to open:

- `server.js` lines 1-100 (Helmet, CORS, rate limiters)
- `supabase/migrations/015_admin_table_and_qls_hardening.sql`
- `features/security/form-validation.js`

Practice task: Read the rate limiter config in `server.js` and understand: how many login attempts are allowed before blocking?

Mistakes to avoid:

- Committing `.env` to Git (NEVER add `.env` to version control)
- Disabling rate limits "just to test" and forgetting to re-enable them

Level 9: Debugging Real Bugs in CLASS-APP

What to study:

- How to use Chrome DevTools (Console, Network, Application tabs)
- How to identify the `pointerEvents` bug described in MUST-FIX
- How to trace "button not working" bugs (is it connected? Is it hidden? Is it blocked?)
- How to read Supabase RLS error messages

Files to open:

- `MUST-FIX` — read the full documented bug analysis
- `script.js` ~line 3197 — find the broken ternary
- `features/hotfixes/interaction-guards.js` — see how guards prevent crashes

Practice task: Open the app, open DevTools Console, navigate to a game page, then navigate back — observe any errors.

Mistakes to avoid:

- Assuming "it works on desktop" means it works on mobile
- Fixing visual bugs without checking if the underlying logic still works

Level 10: Safely Adding New Features

What to study:

- How feature modules are structured (each feature has its own JS/CSS in `features/`)
- How to hook a new page into `goToPage()` in `shell-controls.js`
- How to add a nav item to the sidebar in `index.html`
- How to add a changelog entry in `changelog.js`
- How to bump cache versions in `sw.js` and `index.html`

Files to open:

- `features/personal-tools/personal-tools.js` — a clean example of a module launcher
- `features/logging-in/shell-controls.js` — where to add page teardown/init hooks

- `features/updates/changelog.js` — how to add a new version entry

Practice task: Create a new `features/study-timer/` folder with a `study-timer.js` and a stub HTML section in `index.html`. Wire it into the sidebar nav.

Mistakes to avoid:

- Touching `loading-components.js` or `shell-controls.js` for feature-specific code
- Not testing login, loading screen, and sidebar after any change to shared files
- Skipping the changelog and cache version updates

19. Repository Health Check

What is Well-Built

Area	Status	Details
Security architecture	 Good	JWT auth, bcrypt, RLS policies, rate limiting all in place
Feature modularization	 Good	25 features each in their own folder with isolated JS/CSS
Service worker	 Good	Best-effort caching, clean activation, API routes excluded
AI fallback system	 Excellent	Three-tier fallback (Gemini → Groq → local) is robust
Password security	 Good	bcrypt 12 rounds + legacy SHA-256 upgrade path
Admin system	 Good	Database-driven admin table, no hardcoded usernames
Documentation	 Excellent	CLAUDE.md is extremely detailed; MUST-FIX documents known bugs
File validation	 Good	Type and size checks on uploads
Rate limiting	 Good	Login, AI, and folder operations all rate-limited
Docker support	 Good	Dockerfile included for consistent deployment

What is Confusing

Area	Issue
<code>script.js</code> is 7,571 lines	Extremely long; it mixes page logic, auth, folder system, chat, and more — hard to navigate
Multiple notepad files	<code>features/notepad/notepad.js</code> (legacy) AND <code>features/personal-tools/notepad.js</code> (current) — which is active?
<code>file-summarizer.css</code> in root	There's a CSS file at the root level AND one in <code>features/file-summarizer/</code> — which one is used?
Stub modules	Several feature modules (academics, chat, calendar, etc.) are just 1-2 lines with no logic — the actual code is all inside <code>script.js</code>
Large game files	<code>pokemon.js</code> (161KB) and <code>royale.js</code> (151KB) are very hard to read and maintain
<code>data.json</code>	A JSON file used for runtime data — it's not clear what persists here vs. in Supabase
Subject data hard-coded	Year 3 and Year 4 subjects are empty arrays in <code>script.js</code> — not database-driven

What is Risky

Area	Risk	Recommendation
<code>pointerEvents</code> bug	Buttons on hidden pages receive clicks — causes silent bugs	Fix the ternary in <code>script.js</code> ~line 3197
No virus scanning on uploads	Malicious files could be uploaded	Add server-side file type verification beyond extension check
MIME type validation	Only file extension is checked, not actual MIME type	Use <code>file-type</code> npm package to verify real MIME type
<code>dashboard-mockup.html</code>	HTML file accessible at a public URL — reveals app structure	Add to <code>.gitignore</code> or move to a private docs folder
Ana Cruz demo data	Hardcoded demo name in <code>coding-educational.js</code>	Label clearly as demo/mock data
Large JS files	<code>pokemon.js</code> (161KB) loads for all users even if they never play	Lazy-load game files only when the game page is opened

What Should Be Refactored

Item	Why
<code>script.js</code> (7,571 lines)	Split into domain modules: <code>auth.js</code> , <code>folders.js</code> , <code>chat.js</code> , <code>ui-helpers.js</code> , etc.
Page initialization in <code>goToPage()</code>	Currently a giant if-else chain; should use a page registry pattern
Inline onclick attributes	Migrate to <code>addEventListener</code> for consistency and easier testing
Subject/class data	Move from hard-coded arrays in <code>script.js</code> to the <code>subjects</code> database table
<code>stub</code> feature modules	Either expand them or remove the 1-2 line stubs and move logic directly into proper feature files

What Should Be Documented Better

Item	Recommendation
<code>data.json</code> schema	Document what fields are stored and why
Socket.IO event list	Create a reference of all Socket.IO events (client and server side)
Supabase RLS policies	Add comments inside migration files explaining the intent of each policy
R2 folder structure	Document the exact folder structure inside the R2 bucket
Environment variables	Expand <code>.env.example</code> with descriptions of each variable's purpose

What Files Are Too Large or Overloaded

File	Size	Issue
<code>script.js</code>	7,571 lines / 376KB	Too many responsibilities — needs splitting
<code>changelog.js</code>	137.9KB	Very large; acceptable since it's changelog data
<code>coding-educational-data.js</code>	133.2KB	Large lesson dataset — consider lazy loading
<code>pokemon.js</code>	161.8KB	Entire game in one file — should be lazy-loaded

File	Size	Issue
<code>royale.js</code>	151.3KB	Same issue as pokemon.js

What Security Concerns Exist

Concern	Severity	Status
No MIME type verification on uploads	Medium	Partially mitigated by extension check
Facebook iframe embed	Low	Browser-level risk; Facebook may block it
No malware scanning	Medium	Files are stored and served without content inspection
Service role key must stay server-only	Critical	Currently correct — verify no future change moves it to frontend
JWT in localStorage (vs. httpOnly cookie)	Low-Medium	XSS risk; acceptable tradeoff given CSP headers in place

What Features Are Most Fragile

Feature	Fragility	Reason
Button click handling	High	pointerEvents bug + inline onclick dependency on window globals
Service worker cache	Medium	Version bump discipline required on every change
Lobby multiplayer	Medium	Socket.IO connection depends on server being awake (Render free plan sleeps)
AI summarization	Low	Three-tier fallback makes it resilient
File upload	Medium	Large files (video) may timeout on slow connections

What to Learn First to Maintain This App

Priority order for new maintainers:

- 1. Read **CLAUDE.md** completely — it contains all the rules and protected systems
- 2. Read **MUST-FIX** — understand the documented pointerEvents bug
- 3. Understand the startup sequence — `loading-components.js` → `shell-controls.js` → `script.js`
- 4. Learn the cache version system — never skip bumping versions after changes
- 5. Learn the RLS policies — understand what data each user can access
- 6. Study the folder/file system in `script.js` — it's the most used feature
- 7. Learn how `goToPage()` works — it's the heart of navigation

20. Missing / Unable to Analyze

The following items could not be fully verified from static code analysis:

Item	Reason
Live Supabase table data	Cannot access the live database to verify data integrity
Actual R2 bucket contents	Cannot browse the cloud storage
Push notification delivery	Cannot test Web Push without a browser subscription
iOS Safari behavior	Cannot test on a real iOS device from this analysis

Item	Reason
Socket.IO real-time behavior	Cannot simulate live connections
Render.com deployment behavior	Server cold-start behavior (free plan sleeps after inactivity)
Year 3 and Year 4 subject data	Arrays are empty in <code>script.js</code> — no subjects defined
Gallery feature	<code>features/gallery/gallery.js</code> is a stub with no logic found — may be unimplemented
Event Pictures page	Referenced in navigation but full implementation unclear from stubs
Random Pictures page	Referenced in navigation but full implementation unclear
Social media pages listed	The specific Facebook pages embedded are not visible from the code
Admin diagnostics output	The actual system health data requires a live server

This guide was generated by deeply reading all files in the CLASS-APP repository. Every file name, function name, variable name, line number reference, and code snippet is based on actual code found in the repository — nothing was invented or guessed.

Last analyzed: CLASS-APP v1.6.4 (changelog entry v1.9.16, May 11 2026)